

Міністерство освіти і науки України
ЧЕРКАСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ім. Богдана Хмельницького

Факультет Обчислювальної техніки, інтелектуальних та
управляючих систем
Кафедра Програмного забезпечення автоматизованих систем

ЛАБОРАТОРНА РОБОТА №5
по дисципліні «Системне програмування»

**Тема: МОДЕЛЮВАННЯ І УСУНЕННЯ СИТУАЦІЙ ВЗАЄМНОГО
БЛОКУВАННЯ І ГОЛОДУВАННЯ**

Виконала: студентка гр. КС-24
Гук М.О.

Перевірив: викладач
кафедри ІТ
Авраменко А.С.

Черкаси, 2026

Мета роботи: вивчити принципи виникнення взаємоблокувань і реалізувати моделювання алгоритму банкіра для перевірки безпечності стану системи.

ЗАВДАННЯ

Розробити програму підвищеної складності, що моделює роботу алгоритму банкіра. Кількість процесів має задаватися в межах від 2 до 4, кількість типів ресурсів - від 1 до 3.

- згенерувати випадкову систему процесів і ресурсів у заданих межах;
- сформувати матриці Allocation, Max і Need;
- обчислити вектор доступних ресурсів Available;
- перевірити стан системи алгоритмом банкіра;
- вивести безпечну послідовність або показати, що стан небезпечний;
- виконати запит ресурсів від процесу і перевірити, чи залишиться система в безпечному стані.

ХІД РОБОТИ

Посилання на гіт: https://github.com/HukMariia1124/SystemProgramming_Lab5

Програма реалізована мовою C# з графічним інтерфейсом WPF. Інтерфейс дозволяє генерувати параметри системи, переглядати матриці ресурсів і запускати аналіз алгоритмом банкіра.

У програмі використовуються такі основні позначення:

Таблиця 1 – Поля програми

Позначення	Поле в програмі	Зміст
E	_total	загальна кількість ресурсів у системі
A / Available	_available	кількість ресурсів, які зараз вільні

C / Allocation	_allocation	ресурси, вже виділені процесам
Max	_maximum	максимальні потреби процесів
Need	_need	ресурси, які ще можуть знадобитися процесу
Work	work	робоча копія Available для уявного завершення процесів

Матриця потреб обчислюється за формулою:

```
Need[i, j] = Max[i, j] - Allocation[i, j];
```

Вектор доступних ресурсів обчислюється як різниця між загальною кількістю ресурсу і сумою виділених одиниць цього ресурсу:

```
Available[j] = Total[j] - sum(Allocation[i, j]);
```

Основні можливості інтерфейсу програми:

- випадкова генерація кількості процесів від 2 до 4;
- випадкова генерація кількості типів ресурсів від 1 до 3;
- перегляд і редагування матриць Allocation і Max;
- автоматичний перерахунок Need і Available;
- покроковий лог роботи алгоритму банкіра;
- запит ресурсів від обраного процесу з автоматичною перевіркою безпечності нового стану.

Генерація стану системи виконується в методі GenerateScenario. Спочатку створюється матриця Allocation, після цього для кожного процесу формується Max, а Need рахується як різниця між Max і Allocation.

Скорочений фрагмент генерації стану системи:

```
// Генерація Max і Need після випадкового Allocation.
for (var i = 0; i < processCount; i++)
{
    for (var j = 0; j < resourceCount; j++)
```

```

        {
            var extraNeed = _random.Next(0, Math.Max(2, _total[j] / 2) +
1);
            _maximum[i, j] = Math.Min(_total[j], _allocation[i, j] +
extraNeed);
            _need[i, j] = _maximum[i, j] - _allocation[i, j];
        }
    }
    RecalculateDerivedState();
    BindAllGrids();

```

Перевірка безпечності виконується методом CheckSafety. Алгоритм бере Work = Available і шукає процес, для якого Need не перевищує Work. Якщо такий процес існує, він уявно завершується і повертає виділені ресурси в Work.

```

var work = (int[])_available.Clone();
var finish = new bool[processCount];
var sequence = new List<int>();
while (changed)
{
    changed = false;
    for (var i = 0; i < processCount; i++)
    {
        if (finish[i] || !RowLessOrEqual(_need, i, work))
        {
            continue;
        }
        for (var j = 0; j < resourceCount; j++)
        {
            work[j] += _allocation[i, j];
        }
        finish[i] = true;
        sequence.Add(i);
        changed = true;
    }
}

```

Якщо всі процеси вдалося позначити як завершені, стан вважається безпечним. Якщо на певному кроці немає жодного процесу, для якого виконується $Need \leq Work$, стан є небезпечним.

Окрім перевірки поточного стану, програма підтримує запит ресурсів від конкретного процесу. Метод ResourceRequestButton_Click отримує номер процесу і

вектор запиту, після чого виконує три кроки: перевіряє, що запит не перевищує Need[pid]; перевіряє, що вистачає вільних ресурсів Available; тимчасово виділяє ресурси і запускає CheckSafety. Якщо новий стан безпечний – ресурси виділяються реально; якщо небезпечний – стан відкочується назад і запит відхиляється.

Скорочений фрагмент обробки запиту ресурсів:

```
// Крок 1: запит не перевищує Need[pid]
// Крок 2: є достатньо Available
// Крок 3: тимчасово виділяємо і перевіряємо безпечність
var savedAvailable = (int[])_available.Clone();
var savedAllocation = (int[,])_allocation.Clone();
var savedNeed = (int[,])_need.Clone();

var safety = CheckSafety();
if (!safety.IsSafe)
{
    _available = savedAvailable;
    _allocation = savedAllocation; // відкочуємо стан
    _need = savedNeed;
}
```

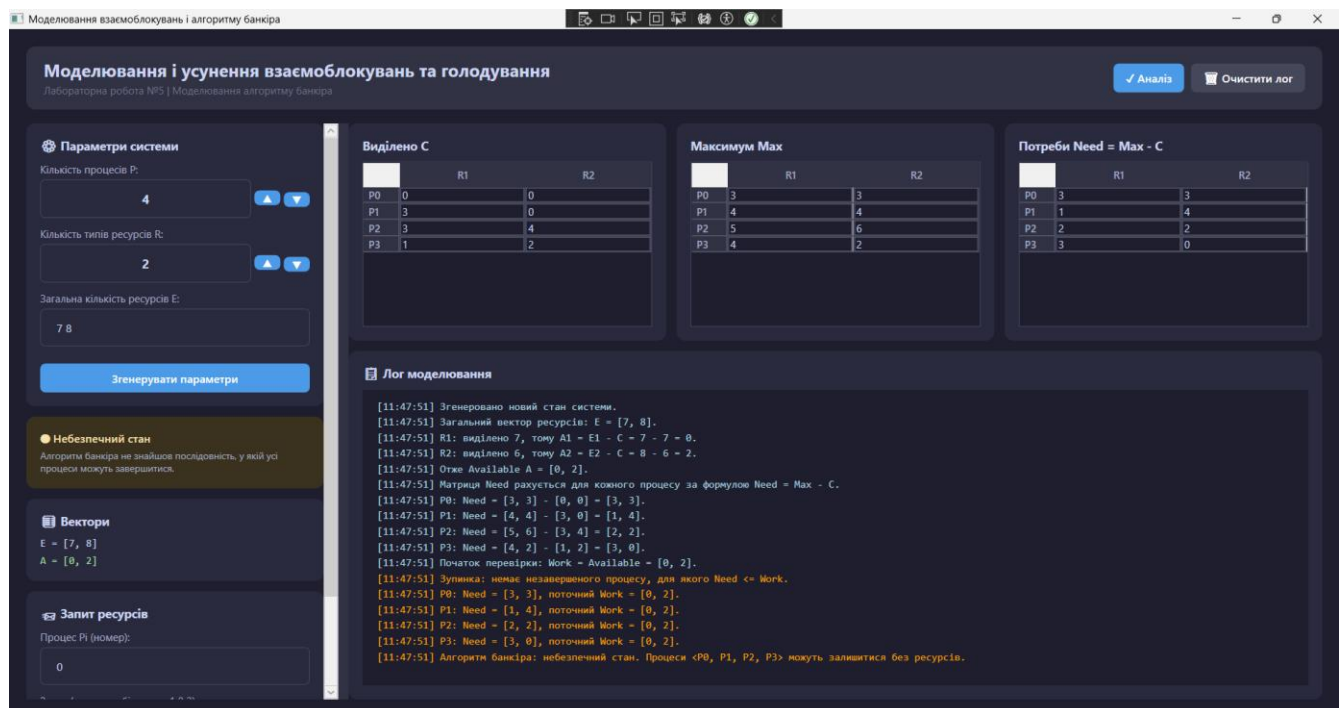


Рис. 1 – Небезпечний стан

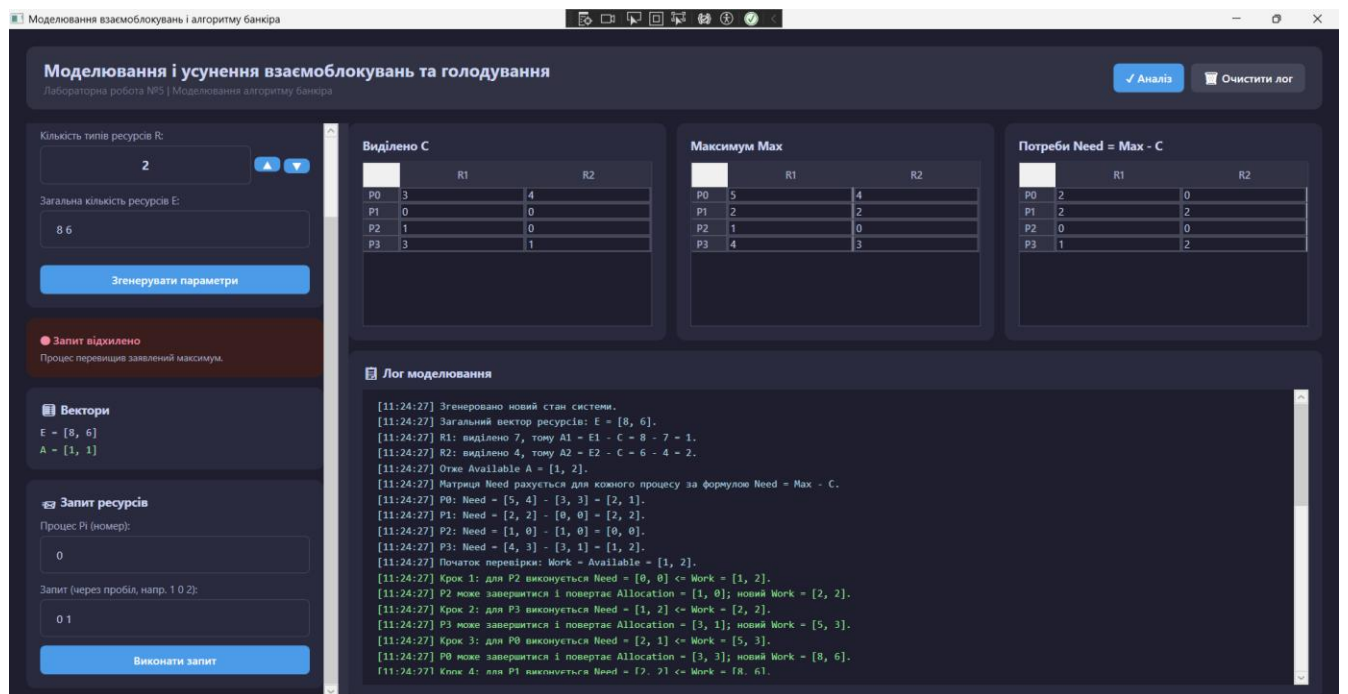


Рис. 2 – Безпечний стан та запити

Лог з прикладу:

```
[11:24:27] Згенеровано новий стан системи.
[11:24:27] Загальний вектор ресурсів: E = [8, 6].
[11:24:27] R1: виділено 7, тому A1 = E1 - C = 8 - 7 = 1.
[11:24:27] R2: виділено 4, тому A2 = E2 - C = 6 - 4 = 2.
[11:24:27] Отже Available A = [1, 2].
[11:24:27] Матриця Need рахується для кожного процесу за формулою
Need = Max - C.
[11:24:27] P0: Need = [5, 4] - [3, 3] = [2, 1].
[11:24:27] P1: Need = [2, 2] - [0, 0] = [2, 2].
[11:24:27] P2: Need = [1, 0] - [1, 0] = [0, 0].
[11:24:27] P3: Need = [4, 3] - [3, 1] = [1, 2].
[11:24:27] Початок перевірки: Work = Available = [1, 2].
[11:24:27] Крок 1: для P2 виконується Need = [0, 0] <= Work = [1, 2].
[11:24:27] P2 може завершитися і повертає Allocation = [1, 0]; новий
Work = [2, 2].
[11:24:27] Крок 2: для P3 виконується Need = [1, 2] <= Work = [2, 2].
[11:24:27] P3 може завершитися і повертає Allocation = [3, 1]; новий
Work = [5, 3].
[11:24:27] Крок 3: для P0 виконується Need = [2, 1] <= Work = [5, 3].
```

[11:24:27] P0 може завершитися і повертає Allocation = [3, 3]; новий Work = [8, 6].

[11:24:27] Крок 4: для P1 виконується Need = [2, 2] <= Work = [8, 6].

[11:24:27] P1 може завершитися і повертає Allocation = [0, 0]; новий Work = [8, 6].

[11:24:27] Алгоритм банкіра: безпечний стан. Можлива послідовність завершення: <P2, P3, P0, P1>. ==

[11:24:33] P0 запитує: [0, 1].

[11:24:33] Крок 1: перевіряємо що Request [0, 1] <= Need[P0] [2, 1].

[11:24:33] Крок 1 пройдено ✓

[11:24:33] Крок 2: перевіряємо що Request [0, 1] <= Available [1, 2].

[11:24:33] Крок 2 пройдено ✓

[11:24:33] Крок 3: тимчасово змінюємо стан системи (уявно виділяємо ресурси):

[11:24:33] Available: [1, 2] → [1, 1]

[11:24:33] Allocation[P0]: [3, 3] → [3, 4]

[11:24:33] Need[P0]: [2, 1] → [2, 0]

[11:24:33] Запускаємо алгоритм безпечності на новому стані...

[11:24:33] Початок перевірки: Work = Available = [1, 1].

[11:24:33] Крок 1: для P2 виконується Need = [0, 0] <= Work = [1, 1].

[11:24:33] P2 може завершитися і повертає Allocation = [1, 0]; новий Work = [2, 1].

[11:24:33] Крок 2: для P0 виконується Need = [2, 0] <= Work = [2, 1].

[11:24:33] P0 може завершитися і повертає Allocation = [3, 4]; новий Work = [5, 5].

[11:24:33] Крок 3: для P1 виконується Need = [2, 2] <= Work = [5, 5].

[11:24:33] P1 може завершитися і повертає Allocation = [0, 0]; новий Work = [5, 5].

[11:24:33] Крок 4: для P3 виконується Need = [1, 2] <= Work = [5, 5].

[11:24:33] P3 може завершитися і повертає Allocation = [3, 1]; новий Work = [8, 6].

[11:24:33] Новий стан безпечний → виділення підтверджується реально.

[11:24:33] Нова послідовність: <P2, P0, P1, P3>.

[11:24:34] P0 запитує: [0, 1].

[11:24:34] Крок 1: перевіряємо що Request [0, 1] <= Need[P0] [2, 0].

[11:24:34] R2: 1 > Need[P0][R2] = 0 – процес перевищив заявлений максимум. Запит відхилено.

ВИСНОВОК

У ході виконання лабораторної роботи було розроблено WPF-програму для моделювання алгоритму банкіра. Програма генерує систему з 2-4 процесів і 1-3 типів ресурсів, формує матриці Allocation, Max і Need, обчислює вектор Available та виконує перевірку стану системи на безпечність. У журналі моделювання покроково відображається робота алгоритму: початковий Work, перевірка умови $Need \leq Work$, уявне завершення процесу та повернення ресурсів. Якщо існує послідовність завершення всіх процесів, стан визначається як безпечний; інакше програма повідомляє про небезпечний стан. Також реалізовано повний алгоритм банкіра для обробки запиту ресурсів: програма тимчасово виділяє запитані ресурси, перевіряє безпечність нового стану і або підтверджує виділення, або відкочує зміни й відхиляє запит.